

Production Hardening

What has to be true before the first kiosk leaves the workshop — ten items agreed 30 September 2026, nine of them free.

Scope farmgirl-books · mbm-checkout **Status** approved, in progress **Rev** V1

Why this addendum exists

On 29 September the Supabase project paused itself. Free-tier projects pause after seven days without activity, and with the stand unplugged and nobody signing in, the clock ran out. Farmgirl Finance was completely unusable for both Carissa and The Coop — and **farmgirl.ai kept returning HTTP 200 the whole time**, because the front end is a static bundle on Netlify that was serving perfectly while the database behind it was switched off. The outage was discovered from a vendor email, not from anything we built.

That prompted a wider question: the product is about to hold several businesses' financial records and take money at unattended stands. This document records what was checked, what was found, and the ten items agreed to close the gap.

Row-level security is correct across all eleven tables. Every tenant-scoped table carries the same policy on both halves — `using` and `with check` — resolving through `entity_id` in `(select id from entities where owner_id = auth.uid())`. The common failure here is declaring `using` alone, which prevents reading another tenant's rows but still permits writing into them. It is absent. Storage buckets for attachments and logos are scoped identically by folder.

Privilege escalation is closed deliberately. `revoke update (is_admin) on profiles from authenticated` stops a signed-in user flipping their own admin bit through the otherwise legitimate "update your own row" policy.

All twelve service-role functions verify the caller before acting. These bypass RLS by design, so the isolation there is hand-written and was the suspected weak point. It is not. Each validates the bearer token against `/auth/v1/user` to obtain a trusted user id, then confirms entity ownership before using the service key. `pos-request` returns 403 "Not your business." `push-subscribe` takes the user id from the verified token and never from the request body. `admin-fleet` reads `is_admin` from the database, refuses before fetching anything, and retrieves counts rather than contents. The two endpoints that accept an id from the request are intentional: `public-products` is the kiosk catalog with cost, sku and stock columns excluded, and `push-send` is server-to-server behind a shared key that can only ever notify the owner of an entity the caller already knew.

Tenant isolation in the accounting app is the strongest part of the system. None of the ten items below concern it.

Blocks handing a stand to a customer

ITEMS 1–3

1 Hours & contact was one record shared by every stand

\$0

FIXED

THE DEFECT — `kiosk-info.mjs` was the last store still writing to a bare blob key (`content`) with no tenant prefix. Every other store had been migrated to `t/<stand>/` during the multi-tenancy work; this one was missed. The Coop saving its opening hours would have overwritten Mini Barn Market's, and each stand would then have displayed the other's on the wall. This was the only confirmed, reproducible cross-tenant defect found anywhere in either codebase.

THE FIX — scoped exactly like `/api/screen` and `/api/montage` : an admin page reads back the stand its session writes to, a kiosk names its own stand in the URL, and an explicit `?stand=` still wins.

`readWithFallback` means Mini Barn Market's existing bare record keeps resolving until its first save moves it, so nothing goes blank on deploy. The store was also switched to strong consistency, matching the fix already applied to screen and montage — without it the `/info` page re-reads stale text straight after saving and a successful save looks broken.

2 A stand off our network cannot be reached at all

\$0

NEEDS ACCOUNT

THE PROBLEM — SSH to a kiosk works only from the same LAN. This is not theoretical: this week our own stand was unreachable from our own house for several days. A kiosk in another owner's barn, behind their router, is a box that cannot be updated, diagnosed or rescued — and the owner has no way to help. Every other item on this list assumes we can reach the hardware.

THE FIX — Tailscale, a private mesh network installed on each kiosk and on the workshop PC. Each machine joins our network regardless of whose router it sits behind, with no port forwarding and nothing exposed to the public internet. The free tier covers 100 devices and 3 users, which is far beyond any plausible fleet. It replaces nothing: LAN SSH keeps working as the fallback.

3 Provisioning a new stand is a sequence of manual steps

\$0

PLANNED

THE PROBLEM — bringing The Coop online meant hand-running SQL, choosing a stand slug, setting environment variables and configuring the box by hand. Every manual step is a step that can be skipped, and a half-provisioned tenant is the most likely way a stand ends up reading someone else's settings. It does not survive customer number five.

THE FIX — one provisioning action, built on the request path that already exists. `pos-request.mjs` already lets an owner request a stand and records it against their entity; `admin-fleet` already lists them. Fulfilment becomes a single admin endpoint that allocates the stand slug, writes the default config records under `t/<slug>/`, registers the device key, links the stand to the entity and returns the finished kiosk URL — succeeding completely or not at all. Onboarding then has one button and no checklist.

Blocks holding paying customers' data

ITEMS 4–5

4 A backup we control — including the one no plan covers

\$0

PLANNED

DOES SUPABASE PRO SOLVE THIS?

Partly, and not the important half. Pro adds daily Postgres backups retained for seven days — genuinely useful, and a real improvement on the free tier, which has **no automatic backups at all**. But those backups live inside the same Supabase account as the database they protect, so a billing lapse, a suspension or a lost login takes both. And Pro does nothing for **Netlify Blobs, which has no backup facility on any plan**. The kiosk's entire sales record lives there under `evt/<date>/`. For a product intended to be as solid as a Square terminal, the transaction log being the least protected thing we own is the wrong way round.

THE FIX — a nightly job that dumps Postgres and exports the blob stores to storage on our own account, with a retention window, and a restore rehearsed once so it is known to work rather than assumed. This doubles as the exit: it is the same operation as moving hosts, which turns "we could be on another database in minutes" from a belief into something verified every night.

5 There is no environment to test in but production

\$0

NEEDS ACCOUNT

THE PROBLEM — this machine has no Node or npm, so nothing can be built or tested locally and a Netlify deploy is the only validation this project has. Every change on this list therefore ships unvalidated to a live application while a customer uses it. This is the root cause beneath several past defects, including a daily-count save bug that shipped twice.

THE FIX — a second Supabase project as staging and a Netlify deploy context pointed at it, so changes are exercised against real infrastructure before customers meet them.

TWO CONSTRAINTS TO EXPECT — the free tier permits two active projects, so staging is the second and it fits exactly. And staging is idle by definition, so it will pause every week and send the same email that prompted this document. That is expected behaviour for a staging project, not a fault.

Needed shortly after launch

ITEMS 6–8

6 An uptime check that signs in and reads a row

\$0

PLANNED

A check that merely requests the home page proves nothing here — farmgirl.ai answered 200 throughout an outage that made it entirely unusable. The check has to authenticate and read an actual record, then alert if it cannot. The same applies to the kiosk: a stand that has stopped sending heartbeats should raise an alert rather than wait to be noticed.

7 Error reporting — there is none today

\$0

NEEDS ACCOUNT

Nothing reports client-side or function errors. The Coop's save failure reached us because a person mentioned it, which is the only channel that currently exists. Sentry's free Developer plan covers this comfortably at our size; the paid Team plan starts around \$26/month and would only become relevant at an error volume we are nowhere near. Exact quotas confirmed at signup.

8 Supabase Pro

\$25/MO

AT LAUNCH

Removes automatic pausing entirely and adds daily backups retained seven days. Priced per organisation with the first project included and additional projects from \$10, so one subscribing client covers the entire backend. Deferred until development ends, which is the right call while no kiosk is deployed — the trigger is the first customer data we cannot afford to lose, not a calendar date.

For context on what was not chosen: the Team plan at \$599/month sells SOC 2, ISO 27001, SSO and priority SLAs. That is compliance procurement, not reliability, and nothing in this market asks for it.

9 Confirm we are outside PCI scope

\$0

PLANNED

One pass to confirm no logged or stored field can ever carry a card number, and that the reader hands back a token rather than a PAN. Staying out of scope is worth far more than filling in a questionnaire, and it is only true if nothing in the checkout log path can ever record one.

10 Terms, privacy, and a client export & delete path

\$0

PLANNED

We hold other businesses' financial records. That needs stated terms, a privacy policy, and a working path for a client to export everything they own and to have it deleted on request. Export is also a selling point rather than an obligation: it is what makes signing up feel reversible.

Accounts to create

GOOGLE SSO ON FARMHOUSEGETAWAYS@GMAIL.COM

SERVICE	FOR	SIGN-IN	COST
Tailscale	Item 2 — reach any stand	Google SSO	Free
Sentry	Item 7 — error reporting	Google SSO	Free
Supabase staging project	Item 5 — existing account, second project	GitHub (already in use)	Free

Cost

ITEM	NOW	AT LAUNCH
Items 1, 3, 4, 6, 9, 10 — our own code	—	\$0
Item 2 — Tailscale, 100 devices	Free tier	\$0
Item 5 — staging project	Free tier	\$0
Item 7 — Sentry Developer	Free tier	\$0
Item 8 — Supabase Pro	Not yet	\$25/mo
Total	\$0	\$25/mo

One client subscription at \$27/month covers the entire production backend, permanently.

Sequencing

Item 5 comes first in practice even though it is listed fifth: staging is what makes the other nine safe to ship, since without it each one is an unvalidated change to a live system. Items 2 and 7 follow as soon as the

accounts exist. Items 1, 3, 4 and 6 are ours to write and will be batched — each production deploy costs 15 credits, so approved changes ship together rather than one at a time. Items 9 and 10 are gated on the card reader and on selling to a customer we do not know personally, in that order.

Item 8 is the only decision left open, and its trigger is the end of development rather than any date.